

Emergency Response Management System

Project Report

OBJECT ORIENTED PROGRAMMING

:Submitted By:

Abdul Rafay Siddique

(B25F0124CS032)

:Instructor:

Ms. Laiba Khalid

Abstract:

Emergency situations such as fires, medical emergencies, and criminal incidents require immediate response from the appropriate emergency services. Traditional emergency response systems often face challenges in efficiently identifying incidents and dispatching the nearest available responder.

The Emergency Response Management System is an Object-Oriented Programming based application developed in C++. The system receives emergency calls, analyzes the incident, determines its type and severity, and dispatches the nearest available responder using a strategy-based dispatch mechanism. The project demonstrates core OOP concepts including encapsulation, inheritance, polymorphism, abstraction, composition, aggregation, and design patterns.

1. Introduction

Emergency response systems play a critical role in public safety by coordinating emergency services such as ambulances, fire trucks, and police units. The objective of this project is to simulate a centralized emergency dispatch center capable of processing emergency calls and assigning the most suitable responder.

The system provides a realistic representation of emergency handling workflows and demonstrates how Object-Oriented Programming principles can be applied to solve real-world problems.

2. Problem Statement

Emergency incidents require rapid and accurate dispatch of response units. In many cases, delays occur because of inefficient allocation of resources and poor incident classification.

The problem addressed by this project is:

"How can an emergency response center automatically analyze emergency incidents and dispatch the nearest appropriate responder using Object-Oriented Programming principles?"

3. Project Objectives

The main objectives of this project are:

- To simulate an emergency response management system.
 - To classify emergency incidents automatically.
 - To determine incident severity levels.
 - To dispatch the nearest suitable responder.
 - To maintain incident analytics and reports.
 - To demonstrate advanced OOP concepts and design patterns.
-

4. Scope of the Project

The system handles:

- Medical emergencies

- Fire emergencies
- Crime-related emergencies

The system performs:

- Emergency call registration
- Incident classification
- Severity determination
- Nearest responder selection
- Dispatch management
- Incident analytics generation

Future enhancements may include database integration, GPS tracking, multi-zone dispatching, and real-time communication systems.

5. System Architecture

The Emergency Response Management System follows a modular architecture consisting of the following major components:

1. Emergency Call Processing Module
2. Incident Analysis Module
3. Dispatch Management Module
4. Responder Management Module
5. Analytics Module
6. Logging Module

Architecture Flow

Emergency Call

↓

Incident Analyzer

↓

Incident Creation

↓

Dispatch Engine

↓

Nearest Unit Strategy

↓

Responder Assignment

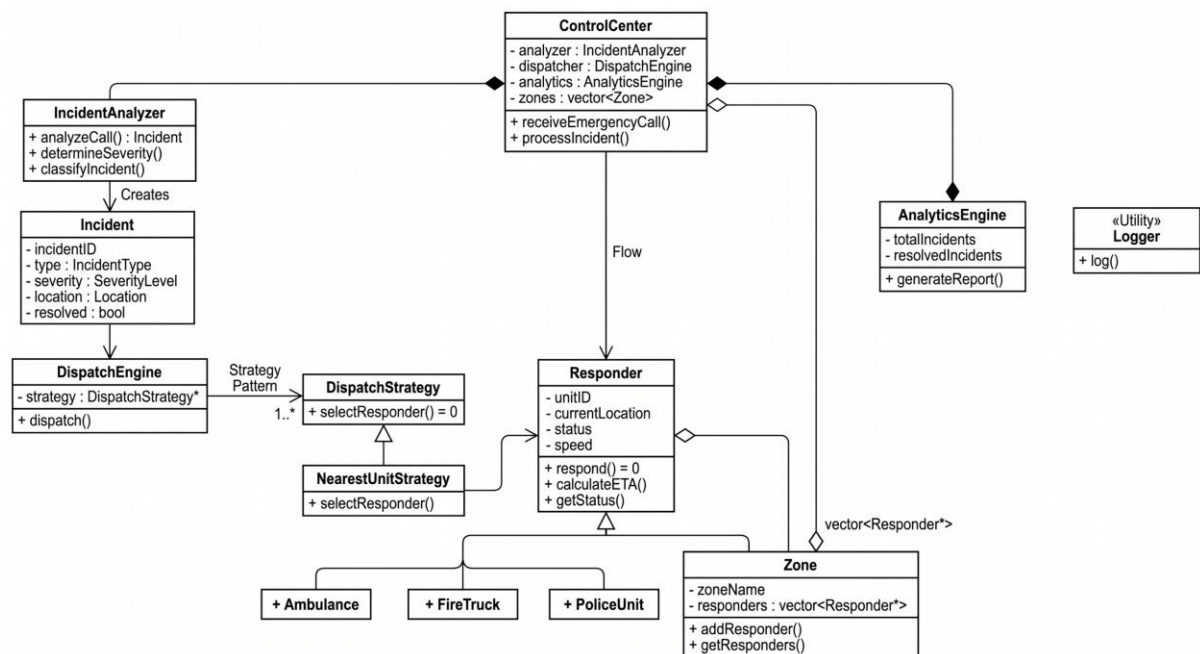
↓

Incident Resolution

↓

Analytics Report

6. UML Class Diagram



The UML diagram illustrates the relationships among system classes and demonstrates inheritance, composition, aggregation, and polymorphism.

7. Algorithms

Algorithm 1: Incident Classification

1. Receive emergency description.
2. Convert description to lowercase.

3. Search for emergency keywords.
4. Determine incident type:
 - Fire
 - Medical
 - Crime
5. Return identified incident type.

Time Complexity

$O(n)$

where n is the length of the description string.

Algorithm 2: Severity Determination

1. Receive emergency description.
2. Search for severity keywords.
3. Determine severity level:
 - Low
 - Medium
 - High
 - Critical
4. Return severity level.

Time Complexity

$O(n)$

Algorithm 3: Nearest Unit Selection

1. Receive incident information.
2. Filter responders by incident type.
3. Calculate distance between responder and incident location.
4. Compare distances.
5. Select responder with minimum distance.

6. Dispatch responder.

Time Complexity

$O(m)$

where m is the number of responders.

8. Class Description

Location Class

Purpose:

Stores geographical coordinates of incidents and responders.

Responsibilities:

- Store X coordinate
 - Store Y coordinate
 - Calculate distance between locations
-

Caller Class

Purpose:

Stores caller information.

Responsibilities:

- Store caller name
 - Store phone number
-

EmergencyCall Class

Purpose:

Represents incoming emergency calls.

Responsibilities:

- Store incident description
 - Store caller details
 - Store location information
-

Incident Class

Purpose:

Represents a classified emergency incident.

Responsibilities:

- Store incident type
 - Store severity level
 - Track incident status
-

IncidentAnalyzer Class

Purpose:

Analyzes emergency descriptions and determines incident type and severity.

Responder Class (Abstract)

Purpose:

Represents emergency response units.

Responsibilities:

- Maintain responder information
 - Provide polymorphic response behavior
-

Ambulance Class

Handles medical emergencies.

FireTruck Class

Handles fire emergencies.

PoliceUnit Class

Handles crime-related emergencies.

DispatchStrategy Class (Abstract)

Purpose:
Provides a generic dispatch strategy interface.

NearestUnitStrategy Class

Purpose:
Selects the nearest responder to an incident.

DispatchEngine Class

Purpose:
Coordinates dispatch operations.

Zone Class

Purpose:
Groups responders into geographical regions.

Logger Class

Purpose:
Maintains system event logs.

AnalyticsEngine Class

Purpose:
Generates performance reports and statistics.

ControlCenter Class

Purpose:
Acts as the central coordinator of the entire system.

9. OOP Concepts Used

Encapsulation

Data members are declared private and accessed through public member functions.

Example:

- Location
 - Caller
 - Incident
-

Inheritance

Responder serves as the base class.

Derived classes:

- Ambulance
 - FireTruck
 - PoliceUnit
-

Polymorphism

A Responder pointer can refer to different responder types and invoke overridden functions at runtime.

Abstraction

The Responder class contains pure virtual functions that define a common interface.

Composition

ControlCenter contains:

- IncidentAnalyzer
 - DispatchEngine
 - AnalyticsEngine
-

Aggregation

Zone maintains references to responder objects.

Strategy Design Pattern

DispatchStrategy and NearestUnitStrategy implement the Strategy Pattern, enabling flexible dispatch algorithms.

10. Testing and Results

Test Case 1

Input:

Critical fire and explosion in warehouse

Expected Result:

FireTruck dispatched

Actual Result:

FireTruck dispatched successfully

Status:

Passed

Test Case 2

Input:

Person unconscious and dying

Expected Result:

Ambulance dispatched

Actual Result:

Ambulance dispatched successfully

Status:

Passed

Test Case 3

Input:

Armed robbery and gun attack

Expected Result:

Police unit dispatched

Actual Result:

Police unit dispatched successfully

Status:
Passed

11. Future Enhancements

Potential improvements include:

- Database integration
 - Real-time GPS tracking
 - Multi-zone dispatching
 - Emergency incident history
 - Priority queues for incidents
 - Multi-agency response dispatching
 - Graphical User Interface (GUI)
 - Web and mobile integration
-

12. Conclusion

The Emergency Response Management System successfully demonstrates the practical application of Object-Oriented Programming concepts to solve a real-world problem. The system analyzes emergency incidents, determines their severity, and dispatches the nearest suitable responder. Through the use of abstraction, inheritance, polymorphism, composition, aggregation, and the Strategy Design Pattern, the project achieves a modular, scalable, and maintainable architecture. The project provides a strong foundation for future development into a fully operational emergency response platform.